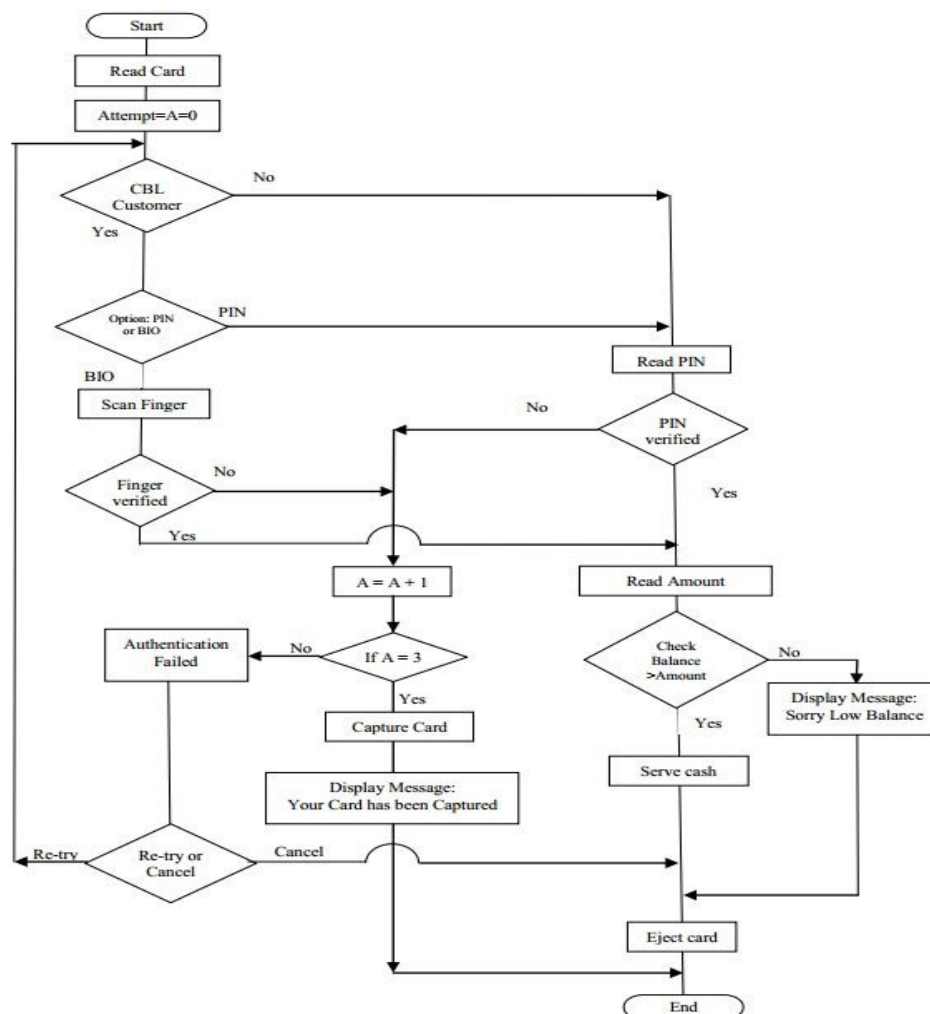


Q1. [1 Mark] You are developing a voice based application that can translate a user's voice samples into a desired language of his/her choice. However, in order to achieve higher accuracy level, you have decided to release a temporary version of the application and based on user feedbacks, you will try to improve the functionalities. You plan to repeat this process till an accuracy level of 90% is reached. Identify the most suitable SDLC model for designing such an application?

Solution: Prototype Model (1 Mark for correct answer, zero otherwise).

Q2.[4 Marks] In addition to traditional PIN control, the ATM machines of Bank of Baroda (BoB) contain fingerprint security features. The customers of BoB have a choice either to use Card + PIN or Card + fingerprint option. Customers of other banks can also withdraw cash from BoB's ATM, but they can only use Card + PIN option. A customer's card is captured by the machine after three consecutive unsuccessful attempts. Describe the aforementioned functionalities using a flowchart showing the process of cash withdrawal from BoB's ATM. Assume that only cash withdrawals are permitted as transactions.

Solution: 1 Mark for checking no.of attempts. 1 Mark for providing both the options to the BoB customers. 2 Marks for overall flow.



Q3. [4 Marks] Write a program in C to display the sum of the series:

$$1 + x + x^2/2! + x^3/3! + \dots$$

Test Data:

Input the value of x :3

Input number of terms : 5

The sum is : 16.375000

Solution: 2 Marks for loop and 2 Marks for overall syntax and program flow.

```
#include<stdio.h>
```

```
Void main()
```

```
{
```

```
    float x,sum,no_row;
```

```
    int i,n;
```

```
    printf("Input the value of x :");
```

```
    scanf("%f",&x);
```

```
    printf("Input number of terms : ");
```

```
    scanf("%d",&n);
```

```
    sum =1;no_row=1;
```

```
    for(i=1;i<n;i++)
```

```
    {
```

```
        no_row=no_row*x/(float)i;
```

```
        sum =sum+no_row;
```

```
    }
```

```
    printf("\n The sum is : %f\n",sum);
```

```
}
```

Q4. [4 Marks] Write a recursive function in C which checks if m is perfect power of b ; i.e, $m=b^k$, where k is a positive integer. m and b are rational numbers represented in the form $m=m_1/m_2$ and $b=b_1/b_2$. m_1, m_2, b_1, b_2 are also positive integers.

Example:

Calling the function with $m=81/256$ and $b=3/4$ should return true.

Calling the function with $m=64/243$ and $b=2/3$ should return false.

Solution: 1 Mark for correct recursive call, 1 Mark each for each base condition and 1 Mark for overall program flow. Please don't award more than 1 mark if solved without recursion.

```
#include <stdio.h>

int if_power(int m1, int m2, int b1, int b2, int M1,int M2)
{   if((m1==M1) &&(m2==M2))
        return 1;
    else if((m1<M1) || (m2<M2))
        return 0;
    else
        return if_power(m1,m2,b1,b2,M1*b1,M2*b2);
}

int main()
{   int m1,m2,b1,b2;
printf("Enter the value of m1,m2,b1,b2\n");
scanf("%d%d%d%d",&m1,&m2,&b1,&b2);
    if(if_power(m1,m2,b1,b2,1,1))
printf("perfect power");
    else
printf("not a perfect power");

    return 0;
}
```

Q5. [6 Marks] Captain Jack Sparrow is anticipating an enemy attack on his vessels and hence wants to destroy the enemy vessels using a specially designed guided missile. His commander alerts him that they have only n (to be taken as input from the user) missiles left in their arsenal. Captain Sparrow fires these missiles, each directed towards co-ordinate (x_i, y_i) , where $1 \leq i \leq n$ and each of x and y are integers. The enemy ship is located in an Amicable coordinate. Captain Sparrow will only succeed only if one of the missiles is directed towards an Amicable coordinate. Create a datatype called location, capable of storing the information about a coordinate and write a C program to test if Captain Sparrow succeeds in destroying the enemy ship with his resources.

A given coordinate (x, y) is termed as an Amicable coordinate if the sum of the proper divisors (not including itself) of x is equal to y and vice – versa. For example $(220, 284)$ is an Amicable coordinate:

First, find the proper divisors of 220:

220:1, 2, 4, 5, 10, 11, 20, 22, 44, 55, 110.

Now add them: $1 + 2 + 4 + 5 + 10 + 11 + 20 + 22 + 44 + 55 + 110 = 284$

Similarly find the proper divisors of 284: 1, 2, 4, 71, 142.

Now add them: $1 + 2 + 4 + 71 + 142 = 220$

***Note:** Captain Sparrow is going to input n distinct coordinates during program execution and if any of the coordinates happens to be an Amicable coordinate, Captain Sparrow succeeds.*

Solution: 1 Mark for correct structure definition, 1 Mark for proper creation of array using DMA, 1 Mark for proper input into the array of structure, 2 Marks for Amicable coordinate testing and 1 Mark for correct output.

```
#include<stdio.h>
#include<malloc.h>

struct location
{
    int x,y;
};

int main(void)
{
    int n,i,j,sum_x,sum_y,flag=0,temp;
    struct location *p;
    printf("\n Enter the number of missiles: ");
    scanf("%d",&n);
    p = (struct location*)malloc(n*sizeof(struct location));
    if(p==NULL)
    {
        printf("Memory allocation failed");
        return 0;
    }
    for(i=0;i<n;i++)
    {
        printf("\n Enter x-coordinate: ");
        scanf("%d",&(p+i)->x);
        fflush(stdin);
        printf("\n Enter y-coordinate: ");
        scanf("%d",&(p+i)->y);
        fflush(stdin);
    }
    printf("\n Entered Coordinates");
    for(i=0;i<n;i++)
    {
        printf("\n x-coordinate: %d", (p+i)->x);
        printf("\n y-coordinate: %d", (p+i)->y);
```

```

}

for(i=0;i<n;i++)
{
    printf("\n Testing for Amicable Coordinate %d & %d",(p+i)->x,(p+i)->y);
    sum_x=0;
    sum_y=0;
    temp = (p+i)->x;
    for(j=1;j<=(temp/2);j++)
    {
        if(temp%j==0)
            sum_x = sum_x + j;
    }
    printf("\n sum_x = %d",sum_x);
    temp = (p+i)->y;
    for(j=1;j<=(temp/2);j++)
    {
        if(temp%j==0)
            sum_y = sum_y + j;
    }
    printf("\n sum_y = %d",sum_y);
    if(sum_x ==(p+i)->y && sum_y==(p+i)->x)
    {
        flag++;
        break;
    }
}
if(!flag)
    printf("\n Failed to destroy enemy ship!!!!");
else
    printf("\n Success!! Destroyed in %d attempt",i+1);
return 0;
}

```

Q6. [4 Marks] A password is considered to be strong if it is an alphanumeric string of length 8 or more and satisfies the following conditions:

- i. The password must comprise of at least one upper case alphabet, one lower case alphabet and one special character.
- ii. The special characters allowed are (@, # and \$).
- iii. The beginning and the end characters cannot be a special character.

Model a function in C that can check the strength of a password as per the above-mentioned rules. Call the designed function via the main function with appropriate argument/arguments and display the password strength.

Solution: 2 Marks for base condition testing and 0.5 marks for each condition checking (i.e, uppercase, lowercase, special character and digits).

```
#include<stdio.h>
#include<string.h>

int testPassword(char *x);

int main(void)
{
    char p[100];

    printf("\n Enter your password: ");
    gets(p);
    fflush(stdin);
    puts(p);
    if(testPassword(p))
        printf("\n Strong Password!!");
    else
        printf("\n Weak Passowrd!!");
    return 0;
}

int testPassword(char *x)
{
    int i,uc=0,lc=0,sc=0,dig=0;
    if(strlen(x)<8)
        return 0;
    else if(!((*x+0)>65 && *x+0)<90) || (*x+0)>65 && *x+0)<90) || (*x+strlen(x))>65 &&
    *x+strlen(x)<90) || (*x+strlen(x))>65 && *x+strlen(x)<90)))
        return 0;
    for(i=0;i<strlen(x);i++)
    {
        if((*x+i)>65 && *x+i)<90))
        {
            uc++;
        }
        else if((*x+i)>97 && *x+i)<122))
            lc++;
        else if ((*x+i)>48 && *x+i)<57))
            dig++;
        else if((*x+i)=='@' || *x+i)=='#' || *x+i)=='$'))
            sc++;
    }
    if(uc > 0 && lc > 0 && dig > 0 && sc > 0)
        return 1;
}
```

```
return 0;
```

```
}
```

Q7. The enrolment number of students of a given institution consists of 12 alphanumeric characters. The first four characters are digits (0000-9999) depicting the year of admission of the student. The next two characters represents his/her department. The next four characters are random digits (0000-9999). The two terminal characters are program code: BT/ MT/ DU/ PH. For example, 2022CS1954BT reveals that the student has taken admission during the year 2022 in the CS department under the BT program. Similarly, 2019EC7826MT reveals that the student has taken admission during the year 2019 in the EC department under the MT program. Develop a C program to perform the following tasks:

- i. **[3 Marks]** Create a text file called "*StudentsInfo.txt*" and store the enrolment number of any number of students onto this file. The program will continue to take input from the user until the user provides a blank input (*i.e.*, the user presses the enter key only).
- ii. **[4 Marks]** Design a function in C that will ask the user to input an enrolment number and check if it is available in the file "*StudentsInfo.txt*". If the enrolment number is available in the file, it will print "Student record Found". Otherwise, the function will write this new enrolment number onto the file "*StudentsInfo.txt*".

Solution: 1 Mark each for opening the file in correct mode (Total 3), 1.5 Marks for correct input from user and writing to file, 1.5 marks for reading from file and testing for availability and 1 Mark for writing back to the file in case the data is missing.

```
#include<stdio.h>
```

```
#include<string.h>
```

```
int main(void)
```

```
{
```

```
FILE *fp;
```

```
char s[15],s1[15];
```

```
int flag=0;
```

```
// Part A:
```

```
fp = fopen("StudentsInfo.txt","w");
```

```
if(fp==NULL)
```

```
{
```

```
printf("\n Unable to open file \n");
```

```
return -1;
```

```
}
```

```
printf("\n enter Enrolment number: ");
```

```
while(strlen(gets(s))!=0)
```

```
{
```

```
fputs(s,fp);
```

```
fputs("\n",fp);
```

```
fflush(stdin);
```

```

    printf("\n enter Enrolment number: ");
}
fclose(fp);

// Part B:

fp=fopen("StudentsInfo.txt","r");
if(fp==NULL)
{
    printf("\n Unable to open file \n");
    return -1;
}
printf("\n enter the enrolment number to be searched: ");
gets(s1);
while(fgets(s,12,fp)!=NULL)
{
    /*printf("\n s=");
    puts(s);
    printf("\t s1=");
    puts(s1);
    printf("\t strcmp = %d",strcmp(s,s1));*/
    if(strcmp(s,s1)==0) // In certain cases it may be -1
        flag++;
}
fclose(fp); // Deduct 1 marks if the file is not closed here before appending.
//printf("\n Flag = %d",flag);
if(!flag)
{
    printf("\n Data unavailable!! Adding to file...\n");
    fp = fopen("StudentsInfo.txt","a");
    if(fp==NULL)
    {
        printf("\n Unable to open file \n");
        return -1;
    }
    fputs(s1,fp);
    fputs("\n",fp);
    fclose(fp);
}
else
{
    printf("Student Record Found");
}

// Testing part not required in actual solution
fp=fopen("StudentsInfo.txt","r");

```

```

if(fp==NULL)
{
    printf("\n Unable to open file \n");
    return -1;
}
while(fgets(s,15,fp)!=NULL)
{

    puts(s);
}
fclose(fp);

return 0;
}

```

Q8. [5 Marks] Write a program in C that will ask the user to input n (to be provided during program execution) integers. There may be duplicate values. Now ask the user to enter a value k and test if the kth largest element is equal to the kth smallest element in the array.

Example:

n	Numbers inputted	k	Output
5	20, 50, 40, 30, 10	3	Equal
6	20, 30, 50, 40, 30, 10	3	Equal
7	20, 30, 50, 40, 30, 10, 25	3	Unequal

Solution: 2 Marks for correct testing as (k-1)th element has to be the (n-k)th element. Please deduct 2 marks for incorrect index testing. 3 marks for overall logic and program flow.

```

#include<stdio.h>
#include<malloc.h>
#include<string.h>

int main(void)
{
    int *arr,n,i,j,k,temp;
    printf("\n Enter the array size: ");
    scanf("%d",&n);
    arr = (int*)malloc(n*sizeof(int));
    if(arr==NULL)
    {
        printf("\n Memory allocation failed");
        return 0;
    }
    for(i=0;i<n;i++)
    {
        printf("\n Enter any integer for [%d] location: ",i+1);
        scanf("%d",&arr[i]);
    }
}

```

```

        fflush(stdin);
    }
    for(i=1;i<n;i++)
    {
        j=i-1;
        temp = arr[i];
        while(j>=0 && arr[j]>temp)
        {
            arr[j+1]=arr[j];
            j--;
        }
        arr[j+1]=temp;
    }
    printf("\n Enter the value of k: ");
    scanf("%d",&k);
    if(arr[k-1]==arr[n-k])
        printf("\n They are equal and the element is %d",arr[k-1]);
    else
        printf("\n Not Equal");
    // the following code is not required:
    printf("\n Sorted Array: \n");
    for(i=0;i<n;i++)
        printf("%d ",arr[i]);

    return 0;
}

```